

Goa RoadSense — Full Collaborative Data Science Project Plan

Project Title

Goa RoadSense: Police-Station and State-Level Analysis of Road Accidents and Traffic Safety in Goa

1. Project Purpose

Goa RoadSense is a collaborative Data Science project that uses official Goa Police road-accident reporting data to investigate accident patterns, fatalities, accident severity, police-station-level differences, traffic enforcement indicators, and the feasibility of advanced/predictive analytics.

The project has **two equally important objectives**:

Objective 1 — Data Science

Build a complete and reproducible workflow:

Official Goa Police Reports



Data Collection



Data Audit



Cleaning & Standardization



Validation



EDA



Statistical Analysis



Feature Engineering



Advanced Analysis



ML Feasibility Evaluation



Final Findings



Dashboard

Objective 2 — Collaboration

Demonstrate genuine collaborative software/data-science development using:

- Git
- GitHub
- Branches
- Issues
- Commits
- Pull Requests
- Peer Reviews
- Merging
- Conflict resolution when naturally required
- Documentation
- Reproducibility

The project should make it possible for an evaluator to inspect GitHub and clearly see that **all four members performed meaningful technical work**.

2. Critical Project Philosophy

The project should **not** be designed around the assumption that an ML model must be produced.

The correct philosophy is:

Use Data Science methods only when the available data supports them.

Therefore:

Data



Can we answer the question?



Can we construct valid features?

↓

Is there enough data?

↓

Is there a valid target?

↓

Can we evaluate the model correctly?

↓

YES → Model

NO → Explain why ML is not justified

A technically honest conclusion such as:

"The available dataset does not provide sufficient observations and target structure for defensible predictive ML."

is better than building an impressive-looking but statistically weak model.

3. Non-Negotiable Data Rules

These rules apply to **all four members**.

Rule 1 — Explicit 00 means zero

If the official source explicitly reports:

00

then:

00 → 0

It must not be converted to missing.

Rule 2 — Missing is NOT zero

If a station/month does not appear in a report:

No record

does **not** mean:

Accidents = 0

It means:

Not observed / unavailable in that report

Do not artificially create zero records.

Rule 3 — YTD is cumulative

YTD values are cumulative values.

For example:

Monthly accidents $\neq$ YTD accidents

YTD fields must retain explicit labels such as:

ytd_accidents

comparison_ytd_accidents

They must never be treated as monthly observations.

Rule 4 — Do not mix analytical grains

The project contains different data grains.

State-level data

Typically:

State $\times$ Year

or

State $\times$ Reporting Period

Used for:

- annual comparisons

- statewide accident trends
- fatalities
- severity
- traffic enforcement
- safety activities

Station-level data

Typically:

Police Station × District × Reporting Period

The available station-level dataset is particularly useful for **December station comparison**.

Therefore:

Statewide annual data and station-level December data must remain separate analytical layers.

Do **not** perform something like:

station data

+

2025 statewide total

just because both have `year = 2025`.

4. What the Project Can and Cannot Claim

We CAN say:

- Accident indicators changed between reported statewide periods.
- Certain accident categories represented larger shares.
- December station-level accident counts differed.
- Some stations had higher observed fatality-to-accident ratios.
- Enforcement indicators changed between available reporting periods.
- Available data supports or does not support particular analytical methods.

We CANNOT automatically say:

- A station is a permanent accident hotspot.

- A station has the highest long-term risk.
 - A person's probability of dying in an accident is $\text{fatalities} / \text{accidents}$.
 - Missing station/month means zero accidents.
 - Enforcement caused accidents to decrease.
 - A machine-learning model predicts future accident risk unless the target, sample, features and evaluation design are genuinely valid.
 - Geographic hotspots exist unless reliable geographic coordinates are available.
-

5. Research Questions

The entire project should revolve around these research questions.

RQ1 — Statewide accident change

How did statewide accident and fatality indicators change between the available 2024 and 2025 reporting periods?

RQ2 — Accident severity

What does the statewide accident-severity structure look like across fatal, grievous, minor-injury and non-injury accidents?

RQ3 — Station-level comparison

How do December accident and fatality measures vary across police stations?

RQ4 — Fatality relative to accident volume

Which December station records show relatively high fatalities compared with their observed accident volume?

This uses a descriptive metric such as:

$\text{fatalities_per_accident} = \text{fatalities} / \text{accidents}$

If:

$\text{accidents} = 0$

then:

$\text{fatalities_per_accident} = \text{undefined}$

Do NOT automatically make it zero.

RQ5 — Traffic enforcement

How did available traffic-enforcement indicators change between reported statewide periods?

Possible indicators include:

- MV cases booked
- fines collected
- licence-related actions
- other available enforcement indicators

Only use fields actually present in the source.

RQ6 — Road-safety activities

What patterns are visible in available road-safety activities across the reported periods?

Do not create artificial monthly trends where the source does not provide continuous monthly coverage.

RQ7 — ML feasibility

Does the available dataset support a defensible predictive machine-learning task?

This is a question to be evaluated, not a predetermined YES.

6. Final Repository Structure

Use this repository structure:

```
goa-roadsense/
├── data/
│   ├── raw/
│   ├── interim/
│   └── processed/
├── notebooks/
│   ├── 01_data_audit_cleaning.ipynb
│   ├── 02_data_validation.ipynb
│   ├── 03_exploratory_analysis.ipynb
│   ├── 04_statistical_analysis.ipynb
│   ├── 05_advanced_analysis.ipynb
│   └── 06_final_analysis.ipynb
├── src/
│   ├── data_utils.py
│   ├── validation.py
│   ├── features.py
│   ├── analysis_utils.py
│   └── modeling.py
├── tests/
│   ├── test_validation.py
│   └── test_features.py
├── dashboard/
│   └── app.py
├── figures/
│   ├── eda/
│   ├── statistics/
│   └── dashboard/
├── docs/
│   ├── data_dictionary.md
│   └── data_quality.md
```

```
|   └─ team_contributions.md
|
| ── README.md
| ── PROJECT_DOCUMENTATION.md
| ── requirements.txt
| ── .gitignore
```

Important

`modeling.py` should only contain actual model code **if ML passes the feasibility gate**.

Do not create fake ML files simply to make the repository look advanced.

7. Exact Division of Work — Four Members

MASTER RESPONSIBILITY MATRIX

Member	Primary Responsibility	Main Notebooks	Main Code	Main Outputs
Member 1	Data Collection, Audit, Cleaning, Validation	01, 02	<code>data_utils.py</code> , <code>validation.py</code>	Clean data, validation, dictionary
Member 2	EDA + Statistical Analysis	03, 04	<code>analysis_utils.py</code>	EDA, statistics, findings
Member 3	Feature Engineering + Advanced Analysis + ML Feasibility	05	<code>features.py</code> , optional <code>modeling.py</code>	Features, advanced analysis, ML decision
Member 4	Visualization + Dashboard + Documentation + Integration	06	<code>dashboard/app.py</code>	Dashboard, final notebook, docs

8. MEMBER 1 — Data Collection, Audit, Cleaning & Validation

Primary Mission

Member 1 owns the **data foundation**.

Nothing downstream should be considered final until Member 1's data handoff is accepted.

8.1 Files Owned by Member 1

notebooks/01_data_audit_cleaning.ipynb

notebooks/02_data_validation.ipynb

src/data_utils.py

src/validation.py

docs/data_dictionary.md

docs/data_quality.md

Member 1 also owns the initial population of:

data/raw/

data/interim/

data/processed/

9. Member 1 — Notebook 01

01_data_audit_cleaning.ipynb

Section 1 — Source inventory

Create a table:

Source file	Period	Type	Parsed ?	Note s
----------------	--------	------	-------------	-----------

Record every official report used.

Section 2 — Schema inspection

Inspect:

- columns
 - data types
 - null values
 - number of records
 - unique stations
 - unique periods
 - numerical ranges
-

Section 3 — Data grain

Explicitly identify:

State-level annual

State-level reporting period

Station-level December

Station YTD

Section 4 — Cleaning

Perform:

- numeric conversion
 - station-name standardization
 - date/period standardization
 - removal of accidental parsing artifacts
 - consistent column naming
-

Section 5 — Source-name preservation

Use:

source_station_name

standardized_station_name

Example:

```
source_station_name = Colvala  
standardized_station_name = Colvale
```

Never overwrite source information silently.

Section 6 — Explicit zero handling

Verify:

"00" → 0

and not:

"00" → NaN

Section 7 — Missingness

Calculate:

- missing columns
- missing station-period combinations
- missing months
- missing values within existing records

Clearly distinguish:

missing value

from:

explicit zero

10. Member 1 — Notebook 02

02_data_validation.ipynb

Validation checks should include:

Check 1

Required columns exist.

Check 2

Numeric accident/fatality values are non-negative.

Check 3

Duplicate station-period records.

Expected uniqueness should be documented.

Check 4

Explicit zeros remain zero.

Check 5

Station names are standardized consistently.

Check 6

YTD fields are not accidentally treated as monthly values.

Check 7

Statewide and station datasets remain separate.

Check 8

Coverage is correctly documented.

11. Member 1 — `src/data_utils.py`

Suggested functions:

```
load_station_data()load_state_data()standardize_station_names()parse_reporting_period()validate_numeric_columns()get_available_periods()
```

Functions should be reusable by downstream notebooks.

12. Member 1 — `src/validation.py`

Suggested functions:

`check_required_columns()`
`check_non_negative_values()`
`check_duplicate_keys()`
`check_explicit_zero_values()`
`check_station_names()`
`check_reporting_periods()`
`generate_validation_summary()`

13. Member 1 — Required Outputs

Member 1 must produce:

`data/processed/monthly_station_accidents.csv`
`data/processed/goa_accident_scenario.csv`

`docs/data_dictionary.md`
`docs/data_quality.md`

and validation outputs.

14. Member 1 — Handoff to Member 2

Member 1 gives Member 2:

1. processed datasets
2. data dictionary
3. data-quality report
4. source coverage
5. list of known limitations
6. validation results

Handoff condition

Member 2 must confirm:

"I have checked the data grain, required columns, zero handling and missingness rules."

Only then does Member 2 finalize analysis.

15. MEMBER 2 — EDA + Statistical Analysis

Primary Mission

Member 2 turns validated data into evidence.

15.1 Files Owned

notebooks/03_exploratory_analysis.ipynb

notebooks/04_statistical_analysis.ipynb

src/analysis_utils.py

figures/eda/

figures/statistics/

16. Member 2 — Notebook 03

03_exploratory_analysis.ipynb

Structure:

1. Objective
2. Data loading
3. Data-grain confirmation
4. State-level EDA
5. Accident severity
6. Fatalities/injuries

7. December station analysis
 8. Traffic enforcement
 9. Safety activities
 10. Key observations
-

17. Member 2 — State-Level EDA

Analyze:

- total accidents
 - fatal accidents
 - grievous accidents
 - minor injury accidents
 - non-injury accidents
 - persons killed
 - injuries
 - rider/pillion/driver/pedestrian/passenger/cyclist outcomes where available
 - enforcement indicators
 - fines where available
-

18. Member 2 — Station-Level EDA

Focus on the available December station data.

Possible visuals:

Bar chart

Accidents by station.

Bar chart

Fatalities by station.

Scatter plot

Accidents

vs

Fatalities

Ratio comparison

`fatalities_per_accident`

Ranking table

Station:

Accidents

Fatalities

Fatalities per accident

But label it:

December station comparison

not:

Goa's permanent accident hotspots.

19. Member 2 — Statistical Analysis

04_statistical_analysis.ipynb

Possible analyses:

Percentage change

`percentage_change =`
 $(\text{new} - \text{old}) / \text{old} \times 100$

Only use it when the denominator and comparison periods make sense.

Severity share

`severity_share =`
`category_count / total_accidents`

Fatalities per accident

```
fatalities_per_accident =  
fatalities / accidents
```

Handle:

```
accidents = 0
```

```
as undefined.
```

20. Important Statistical Limitation

Do NOT pretend that two annual statewide observations are a large statistical sample.

For example, do not blindly perform:

t-test
p-value
confidence interval

and present it as strong evidence from only two annual observations.

Instead emphasize:

- descriptive comparison
- percentage changes
- distributions where enough station observations exist
- descriptive ratios
- variation
- effect sizes where appropriate
- limitations

If an inferential test is used, explicitly explain:

1. unit of analysis
 2. sample size
 3. assumptions
 4. why the test is appropriate
-

21. Member 2 — `analysis_utils.py`

Possible functions:

`calculate_percentage_change()``calculate_severity_share()``calculate_fatality_ratio()``summarize_by_station()``summarize_by_period()``create_comparison_table()`

22. Member 2 — Required Output

Member 2 must deliver:

EDA notebook
Statistical analysis notebook
EDA figures
Statistical figures
RQ findings table

The findings table should look like:

Research Question	Metric	Finding	Caveat	Figure
-------------------	--------	---------	--------	--------

23. Member 2 → Member 3 Handoff

Member 2 tells Member 3:

- which features are meaningful
- which variables are reliable
- which variables have problematic missingness
- which grain each variable belongs to
- which research questions remain unanswered
- what advanced analysis may be useful

Member 3 must confirm:

"I understand the analytical grain and will not create leakage or invalid cross-grain features."

24. MEMBER 3 — Feature Engineering + Advanced Analysis + ML Feasibility

Primary Mission

Member 3 is **not automatically** the "ML person."

Member 3's real job is:

Determine whether advanced analytics and ML are actually justified.

25. Member 3 — Files

notebooks/05_advanced_analysis.ipynb

src/features.py

tests/test_features.py

src/modeling.py

`modeling.py` is optional.

26. Member 3 — Feature Engineering

Potential features:

Fatalities per accident

`fatalities_per_accident = fatalities / accidents`

with zero denominator handled safely.

Severity share

fatal_sharegrievous_shareminor_sharenon_injury_share

Accident change

accident_change_pct

Fatality change

fatality_change_pct

Station descriptive ranks

station_accident_rank

station_fatality_rank

These are **December descriptive rankings**.

They must not be interpreted as permanent risk rankings.

27. Feature Engineering Rules

Every feature must have:

Feature name

Definition

Source columns

Formula

Analytical grain

Missing-value rule

Zero-denominator rule

Interpretation

Example:

Feature	Formula	Grain	Zero rule
fatalities_per_accident	fatalities / accidents	Station-period	undefined if accidents=0
severity_share	category / total	State-period	undefined if total=0

28. Advanced Analysis Options

Possible advanced analysis:

Option A — Station profiling

Group December station records based on:

- accident volume
 - fatalities
 - fatality ratio
 - severity indicators
-

Option B — Clustering

K-Means can be considered if the feature matrix is sufficiently informative.

Possible input:

accidents
fatalities
severity shares
fatality ratio

But the conclusion must be:

"These stations form similar profiles within the available December observations."

NOT:

"These are Goa's permanent high-risk zones."

29. ML Feasibility Gate

Before writing a model, Member 3 must answer:

Question 1

Do we have enough observations?

Question 2

Do we have a meaningful target?

Question 3

Can the target be predicted from information available before the outcome?

Question 4

Are the predictors valid?

Question 5

Is there data leakage?

Question 6

Can we perform proper train/test validation?

Question 7

Is the target sufficiently varied?

Question 8

Can the model be meaningfully evaluated?

30. ML Decision Tree

Is there a valid prediction target?

NO → No ML

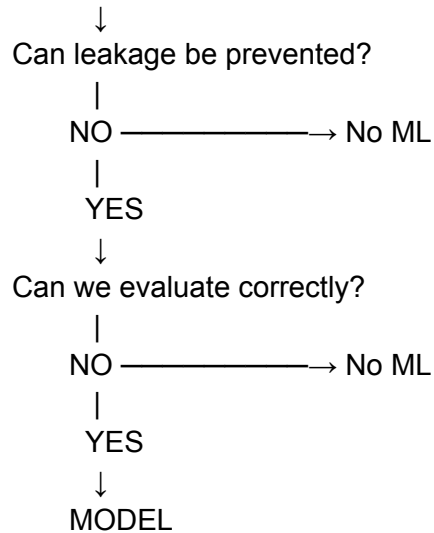
YES

↓

Are there enough observations?

NO → No ML

YES



31. If ML Is Not Justified

The project should explicitly state:

"The available dataset was evaluated for predictive machine learning, but the available observation count, target structure, temporal coverage and/or evaluation constraints did not support a defensible predictive model. Therefore, the project prioritizes descriptive and statistical analysis."

This is completely acceptable.

32. If ML Is Justified

Then Member 3 owns:

src/modeling.py

and must document:

- target
- predictors
- train/test split
- preprocessing
- baseline

- model
- evaluation metric
- cross-validation if appropriate
- leakage prevention
- feature importance/explainability
- limitations

Do not report accuracy alone.

33. Member 3 — Tests

`tests/test_features.py`

Test:

```
test_fatality_ratio()
test_zero_accident_ratio()
test_severity_share()
test_percentage_change()
```

Example expected behavior:

```
accidents = 0
fatalities = 0
```

```
ratio = NaN
```

not:

```
ratio = 0
```

34. Member 3 → Member 4 Handoff

Member 3 gives Member 4:

```
feature definitions
advanced-analysis results
cluster results if used
ML feasibility decision
```

model results if valid
limitations
visualization recommendations

Member 4 must not invent additional ML claims in the dashboard.

35. MEMBER 4 — Visualization + Dashboard + Documentation + Integration

Primary Mission

Member 4 converts the team's analytical work into a coherent final product.

Member 4 is also the **integration owner**, not the owner of everyone else's analysis.

36. Member 4 — Files

dashboard/app.py

notebooks/06_final_analysis.ipynb

README.md

PROJECT_DOCUMENTATION.md

docs/team_contributions.md

figures/dashboard/

37. Dashboard Structure

Page 1 — Overview

Show:

- annual accident indicators
- fatalities
- major comparison metrics
- high-level findings

Do NOT put December station totals into annual KPI cards.

38. Page 2 — Accident Analysis

Show:

- accident severity
 - fatal accidents
 - grievous accidents
 - minor injury accidents
 - non-injury accidents
 - persons killed/injured
-

39. Page 3 — Station Analysis

Show:

- station accident comparison
- station fatality comparison
- accidents vs fatalities
- descriptive fatality ratio
- station table

Heading must clearly indicate:

December Station Analysis

40. Page 4 — Traffic Safety

Show available:

- MV cases
- fines
- enforcement indicators
- safety activities
- related available indicators

Use their actual reporting period.

41. Page 5 — Advanced Analytics

If M3 produces valid advanced analysis:

Show:

- feature profiles
- clusters
- model results
- feature importance

If ML is not justified:

Show:

ML Feasibility Assessment

and explain why predictive modeling was not defensible.

Do NOT manufacture an ML model.

42. Visualization Standards

Every chart must have:

- meaningful title
- axis labels
- units
- time period
- data grain
- source/context

- readable labels

Avoid:

- decorative charts
 - 3D charts
 - misleading scales
 - unsupported geographic maps
 - charts that do not answer a research question
-

43. 06_final_analysis.ipynb

This is the final clean notebook.

Structure:

1. Project objective
2. Research questions
3. Data source
4. Data coverage
5. Data validation
6. State-level analysis
7. Station-level December analysis
8. Feature engineering
9. Advanced analysis
10. ML feasibility
11. Key findings
12. Limitations
13. Conclusion

This should **not** contain every experiment ever performed.

It should tell the final analytical story.

44. README.md

README should contain:

Project title
Project description
Research questions
Dataset/source
Data coverage
Important data limitations
Repository structure
Installation
How to run notebooks
How to run dashboard
Team members
Git collaboration workflow
Key findings
ML feasibility
Limitations

45. PROJECT_DOCUMENTATION.md

This should contain the detailed methodology:

1. Introduction
 2. Problem statement
 3. Objectives
 4. Research questions
 5. Data sources
 6. Data architecture
 7. Data cleaning
 8. Validation
 9. EDA
 10. Statistical analysis
 11. Feature engineering
 12. Advanced analysis
 13. ML feasibility
 14. Dashboard
 15. Git/GitHub collaboration
 16. Limitations
 17. Final conclusions
-

46. docs/team_contributions.md

Record actual contributions.

Example:

Team Contributions

Member 1

- Source inventory
- Data cleaning
- Validation functions
- Data dictionary
- Peer review of dashboard integration

Member 2

- EDA
- Statistical analysis
- Research-question mapping
- Peer review of feature engineering

Member 3

- Feature engineering
- Advanced analysis
- ML feasibility evaluation
- Feature tests
- Peer review of EDA

Member 4

- Dashboard
- Final notebook
- Documentation
- Integration
- Peer review of data pipeline

Do not fabricate contributions.

47. Git Branch Strategy

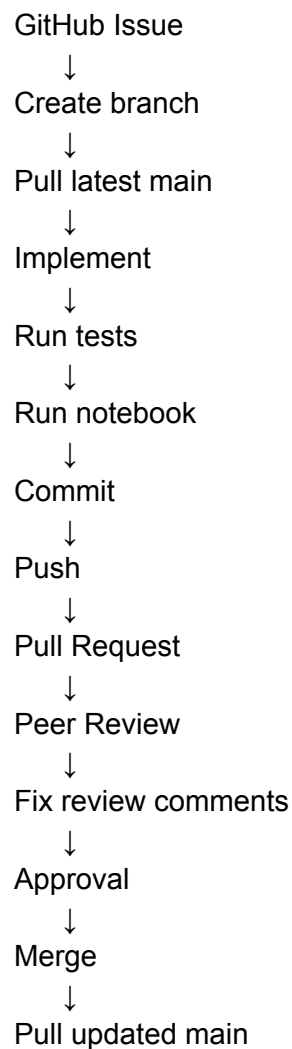
Use:

main
|

- feature/data-engineering
 - feature/eda-statistics
 - feature/advanced-analysis
 - feature/dashboard-documentation
-

48. Git Workflow

Every major task:



49. Example GitHub Issues

Issue #1

Title:

Audit official Goa Police accident reports

Owner:

Member 1

Deliverable:

Source inventory and audit notebook

Issue #2

Title:

Create validated station-level dataset

Owner:

Member 1

Deliverable:

Processed station CSV

Issue #3

Title:

Perform statewide accident EDA

Owner:

Member 2

Issue #4

Title:

Analyze December station differences

Owner:

Member 2

Issue #5

Title:
Build accident/fatality derived features

Owner:
Member 3

Issue #6

Title:
Evaluate ML feasibility

Owner:
Member 3

Issue #7

Title:
Build dashboard

Owner:
Member 4

Issue #8

Title:
Integrate final notebook and documentation

Owner:
Member 4

50. Realistic Commit Examples

Member 1

feat: add source report inventory
feat: standardize police station names
fix: preserve explicit zero values
feat: add reporting period fields

test: add station duplicate validation
docs: document data coverage

Member 2

feat: add statewide accident severity EDA
feat: add December station comparison
feat: add descriptive fatality ratio
feat: add enforcement analysis
docs: map findings to research questions

Member 3

feat: add accident fatality features
test: handle zero accident denominator
feat: create station feature matrix
feat: evaluate clustering feasibility
docs: record ML feasibility decision

Member 4

feat: add dashboard overview
feat: add accident analysis page
feat: add station comparison page
feat: add final analysis notebook
docs: add project methodology
docs: update team contributions

51. Commit Rules

Avoid:

update
changes
final
done
new
test
asdf

Commit messages should explain the change.

Bad:

update files

Good:

fix: handle zero denominator in fatality ratio

52. Pull Request Requirements

Every PR should contain:

What changed?

Why?

Related issue

Files changed

How was it tested?

Output / screenshots

Known limitations

Reviewer

53. Peer Review Structure

Use:

M1 → reviewed by M2

M2 → reviewed by M3

M3 → reviewed by M4

M4 → reviewed by M1

This creates a genuine review cycle.

54. Natural Conflict Resolution

Do NOT create fake conflicts.

If a real conflict occurs:

Pull latest main

↓

git merge main

↓

Identify conflict

↓

Discuss correct version

↓

Resolve conflict

↓

Run tests

↓

Commit resolution

↓

Push

↓

Continue PR

Document only conflicts that actually occurred.

55. Shared Integration Work

All members participate in:

Final notebook review

Everyone checks:

Data

↓

EDA

↓

Statistics

↓

Features

↓
Advanced analysis
↓
Findings

Dashboard verification

Everyone checks dashboard numbers against notebooks.

Reproducibility test

At least one final clean run should be performed.

Presentation rehearsal

All members participate.

56. Timeline

Phase 1 — Repository Setup

All members

Create:

GitHub repository
README
.gitignore
folder structure
Git branches
GitHub Issues

57. Phase 2 — Data Foundation

Member 1

Complete:

data audit
cleaning
validation
data dictionary
data quality

Member 2

Reviews Member 1.

Member 3

Defines feature requirements.

Member 4

Prepares documentation/dashboard requirements.

58. Phase 3 — Core Analysis

Member 2

Completes:

EDA
statistics
research-question findings

Member 1

Supports data questions.

Member 3

Reviews analytical assumptions.

Member 4

Begins visualization planning.

59. Phase 4 — Advanced Analysis

Member 3

Completes:

feature engineering
advanced analysis
ML feasibility

If ML is justified:

model
evaluation
explainability

If not:

ML feasibility report

60. Phase 5 — Dashboard and Integration

Member 4

Completes:

dashboard
final notebook
README
documentation
team contribution file

Other members review.

61. Phase 6 — Final QA

All members:

pull main

↓

run tests

↓

run final notebook

↓

verify dashboard

↓

check documentation

↓

check Git history

↓

presentation rehearsal

62. Testing Requirements

Minimum tests:

test_required_columns

test_non_negative_accidents

test_non_negative_fatalities

test_duplicate_station_periods

test_explicit_zero_handling

test_zero_denominator_ratio

test_severity_share

63. Reproducibility Requirements

Before final submission:

- requirements.txt updated
- raw data preserved
- processed data reproducible
- notebooks execute
- functions execute

- tests pass
 - dashboard runs
 - README instructions work
 - no absolute local file paths
 - no hidden manual transformations
 - no undocumented preprocessing
 - no missing dependencies
-

64. Final Colab Requirement

The final Google Colab notebook should contain:

Project introduction

↓

Research questions

↓

Install/import dependencies

↓

Load repository/data

↓

Data validation

↓

EDA

↓

Statistical analysis

↓

Feature engineering

↓

Advanced analysis

↓

ML feasibility

↓

Final findings

The final Colab is the **clean reproducible demonstration**, not the place where random experimentation is dumped.

65. Presentation Division

Member 1 — Data

Explain:

- source
 - collection
 - cleaning
 - validation
 - missing data
 - zero handling
 - YTD handling
 - limitations
-

Member 2 — Analysis

Explain:

- research questions
 - EDA
 - statewide findings
 - station findings
 - statistical reasoning
-

Member 3 — Advanced Analytics

Explain:

- feature engineering
 - derived metrics
 - advanced analysis
 - ML feasibility
 - whether ML was used and why
-

Member 4 — Product

Explain:

- dashboard
 - final notebook
 - documentation
 - GitHub collaboration
 - integration
-

Everyone

Must be able to answer:

Why did you separate state-level and station-level data?

Why isn't missing data treated as zero?

Why isn't December station data called a permanent hotspot analysis?

Why did you use or reject ML?

How did GitHub demonstrate collaboration?

66. Final Quality Checklist

DATA

- Raw data preserved
 - Source files documented
 - Data dictionary completed
 - Explicit 00 preserved as zero
 - Missing $\neq$ zero
 - YTD correctly handled
 - Station names standardized
 - Duplicate records checked
 - Data grain documented
-

EDA

- RQ1 answered
 - RQ2 answered
 - RQ3 answered
 - RQ4 addressed
 - RQ5 addressed
 - RQ6 addressed
 - Unsupported questions documented
-

STATISTICS

- Correct units of analysis
 - Appropriate calculations
 - No inappropriate significance testing
 - Ratios handled safely
 - No causal claims without causal design
-

FEATURES

- Every feature documented
 - Formula documented
 - Grain documented
 - Missing-value behavior documented
 - Zero denominator handled
 - No leakage
-

ML

- ML feasibility evaluated
 - Target defined
 - Sample size considered
 - Leakage checked
 - Evaluation strategy justified
 - Model only built if justified
-

DASHBOARD

- Overview
 - Accident analysis
 - Station analysis
 - Traffic safety
 - Advanced analytics/ML feasibility
 - Correct labels
 - Correct grain
 - No misleading hotspot claims
-

GITHUB

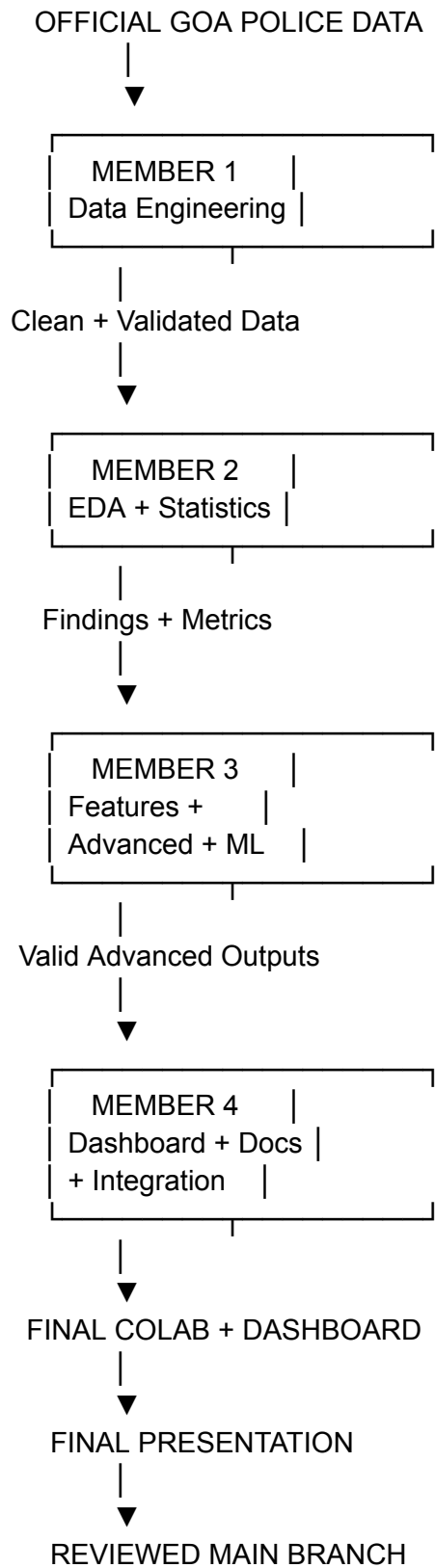
- Four branches used appropriately
 - Meaningful commits
 - GitHub Issues
 - Pull Requests
 - Peer reviews
 - Review comments addressed
 - Merges documented
 - No fake commits
 - No fake conflicts
 - All four members have meaningful technical contributions
-

DOCUMENTATION

- README complete
 - PROJECT_DOCUMENTATION complete
 - Data dictionary complete
 - Data-quality documentation complete
 - Team contributions accurate
 - Limitations documented
-

67. Final Project Workflow

The entire project should ultimately follow:



68. Final Team Rule

The four members are **not** doing four separate projects.

The dependency is:

$M1 \rightarrow M2 \rightarrow M3 \rightarrow M4$

with continuous peer review:

$M1 \leftrightarrow M2$

$M2 \leftrightarrow M3$

$M3 \leftrightarrow M4$

$M4 \leftrightarrow M1$

The strongest version of this project is therefore **not**:

"We made an accident prediction ML model."

It is:

"We built a reproducible collaborative Data Science pipeline using official Goa Police data, rigorously separated incompatible data grains, analyzed accident and traffic-safety patterns, engineered defensible features, evaluated advanced/ML methods against the actual data limitations, and delivered the findings through a reproducible dashboard and GitHub workflow."

That positioning keeps the project technically honest while still demonstrating **data engineering + EDA + statistics + feature engineering + advanced analytics + software collaboration + visualization + reproducibility**.